



Microsoft Defender for Endpoint Connector — Credential Provisioning Guide

Version 0.1.0, 2026-07-15

Table of Contents

Overview.....	1
What kind of account to create.....	1
Permissions to grant	1
Generating the credential.....	3
Mapping values into Crogl.....	3
Verifying the setup.....	3

Overview

The Crogl Defender connector reads security alerts ([GET /api/alerts](#)) and onboarded-device inventory ([GET /api/machines](#)) directly from the Microsoft Defender for Endpoint API at [api.securitycenter.microsoft.com](#). It authenticates as an application, not as a signed-in user, using the OAuth2 client-credentials grant—so there is no interactive login step and no per-analyst account. Everything in this guide happens once, in the customer’s own Microsoft Entra ID tenant, before the connector is configured in Crogl.

These steps require **Application Administrator** or **Global Administrator** access in the customer’s tenant, plus a role that can grant admin consent (Global Administrator or Privileged Role Administrator). If you’re an FDE walking a customer through this live, expect their tenant admin to drive the Entra admin center while you confirm each value.

NOTE

This is a Defender for Endpoint tenant requirement, not a Crogl requirement: Defender for Endpoint must already be provisioned and have at least one onboarded device for the [machines](#) resource to return data (see [Verifying the setup](#) for what an empty device inventory actually looks like on this API).

What kind of account to create

Create an **app registration** (a service principal), not a user account.

1. Sign in to the [Microsoft Entra admin center](#) as a Global Administrator or Application Administrator.
2. Go to **Identity > Applications > App registrations > New registration**.
3. Name it something identifiable, e.g. [Crogl Defender Connector](#).
4. Supported account type: **Single tenant** (accounts in this organizational directory only).
5. Leave **Redirect URI** blank—the client-credentials grant is unattended and never redirects a browser.
6. Select **Register**.

After registration, the app’s **Overview** page shows two of the three values this guide ends with: the **Application (client) ID** and the **Directory (tenant) ID**.

Permissions to grant

WARNING

Unlike a Microsoft Graph-based connector (Intune, for example), the Defender for Endpoint API is **not** listed under Microsoft Graph in the API permissions picker, and it does not show up if you just scroll the default list. It’s a separate API registration Microsoft calls [WindowsDefenderATP](#), and you have to search for that exact name to find it. This is the single most common setup mistake for this connector—admins who are used to adding Graph permissions look in the wrong place and conclude the permission doesn’t exist.

1. In the app registration, go to **API permissions** > **Add a permission** > **APIs my organization uses**.
2. Search for **WindowsDefenderATP** (not "Defender", not "Microsoft Graph") and select it.
3. On the permission-type screen, select the **Application permissions** tile — not Delegated. There is no signed-in user in this flow (client-credentials grant), so a Delegated grant will never work here.
4. Add both of the following Application permissions — this connector is read-only (**GET** only), so start with the read-only grant for each:

Permission	Why
Alert.Read.All	For the GET /api/alerts (list) and GET /api/alerts/{id} (get-by-id) endpoints this connector calls. This is the least-privileged grant — request it first, not Alert.ReadWrite.All , for a connector that never writes.
Machine.Read.All	For the GET /api/machines endpoint. Machine.ReadWrite.All also works but is unnecessarily broad for a connector that only reads.

WARNING

Microsoft's own documentation disagrees with itself here. The **List alerts** and **Get alert information by ID** API reference pages list only **Alert.ReadWrite.All** in their Application-permission table for these two endpoints—but a live **403** error body from the same underlying API, reported in a Microsoft Q&A thread, reads "**API required roles: Alert.Read.All,Alert.ReadWrite.All**", i.e. the server itself treats them as alternatives. Grant **Alert.Read.All** first. **Only add Alert.ReadWrite.All if calls still 403 after confirming everything else in Verifying the setup is correct** — don't jump straight to the broader grant because one doc page lists it as the sole option.

NOTE

Defender's permission names already distinguish Application from Delegated by suffix (**Alert.Read.All** vs **Alert.Read**; **Machine.Read.All** vs **Machine.Read**) — unlike Intune's Graph permission, there is no identically-named Delegated/Application pair to mix up here. The failure mode on this connector is the wrong-API-surface trap above, not a wrong-type-same-name trap.

5. Select **Add permissions**.
6. Back on the API permissions page, select **Grant admin consent for <tenant name>** and confirm.

The permissions show a status of **Granted** once consent succeeds. Skipping admin consent is the second most common setup mistake here — the API returns **HTTP 403** on every call until it's granted, even when the correct Application-type permissions are already added.

NOTE

This connector only reads **alerts** and **machines** today. If a future resource needs a different permission (e.g. an **indicators** or **recommendations** surface), add it under the

same `WindowsDefenderATP` API and re-grant consent — don't over-grant permissions the connector doesn't use.

Generating the credential

1. In the app registration, go to **Certificates & secrets > Client secrets > New client secret**.
2. Add a description (e.g. `crogl-connector`) and an expiry consistent with the customer's credential-rotation policy.
3. Select **Add**.
4. **Copy the secret's Value immediately** — Entra shows it once, at creation time, and never again. If it's missed, delete the secret and generate a new one; there is no way to retrieve a lost value.

Mapping values into Crogl

Three values from the steps above map directly to the `credential_schema.fields` in `community/defender/connector.json` when creating the connector instance in Crogl:

Crogl field	Where it came from	Notes
<code>tenant_id</code>	App registration Overview > Directory (tenant) ID	Identifies the customer's Entra tenant.
<code>client_id</code>	App registration Overview > Application (client) ID	Identifies this specific app registration.
<code>client_secret</code>	Certificates & secrets > the secret Value copied at creation	Treat as a password. Never paste it into chat, a ticket, or this document.

Crogl exchanges these server-side for a bearer token via `https://login.microsoftonline.com/<tenant_id>/oauth2/v2.0/token` (client-credentials grant, `https://api.securitycenter.microsoft.com/.default` scope) and attaches it automatically on every API call — no further action needed once the three values are entered.

Verifying the setup

Before handing credentials to Crogl, or right after creating the connector instance, confirm the setup is correct:

- **In Entra:** API permissions shows both `Alert.Read.All` and `Machine.Read.All` under the `WindowsDefenderATP` API (not Microsoft Graph), **Type** = **Application** for both, and **Status** = **Granted for <tenant>**.
- **In Crogl:** run a query against the connector (e.g. `defender.py query --resource machines --filter "" --limit 1`) and confirm an `HTTP 200` response.
- **If you get 401:** the client ID, tenant ID, or secret value is wrong, or the secret has expired — re-check each value against the app registration.

- **If you get 403 on alerts specifically:** first confirm the permissions were actually added under `WindowsDefenderATP` and not a similarly-named Microsoft Graph permission, and that admin consent was granted. If both check out and `alerts` still 403s with `Alert.Read.All` granted, escalate to `Alert.ReadWrite.All` (see the warning in [Permissions to grant](#)) — Microsoft’s own API reference is inconsistent about which one this specific endpoint accepts.
- **If you get 403 on machines:** check the API column and admin consent per above — there’s no known Read-vs-ReadWrite ambiguity on this endpoint, so a persistent 403 here points to a setup mistake, not a permission-naming quirk.
- **Machines returning 404 on a freshly onboarded tenant is expected, not a connector bug:** Microsoft’s List machines API documents `404 Not Found` (not an empty `200`) when there are no recent machines to report — this differs from the `alerts` endpoint, which returns `200` with an empty `value` array when there are no alerts. If `machines` 404s on a brand-new tenant, wait for at least one device to onboard and report in before treating it as a credential or permission problem.